EAGLE 當前實作流程

Current implementation workflow

1

初始候選 (Seed Candidates)

strategy_prompt / previous_code / generation prompt

2

父代選擇 (Lexicase)

依 7 個 opponent 分數選父代

3

交叉與突變

uniform crossover + strategy / code mutation

4

Java Agent 生成

產生完整 CandidateAgent.java

Strategy Reflection：coverage-aware，最多 10 場 Commentator → Coach

5

驗證與編譯

validation → javac → integration

6

MicroRTS 評估

7 opponents × 3 maps × 3 rounds × p0/p1 = 126 matches

7

Fitness

7 個 opponent scores（主目標）

8

生存者選擇

offspring-first lexicase → 下一代

無 explicit elitism；舊 parent 只在 offspring 不足時補入

Lexicase Selection 怎麼做

Active parent selection in current EAGLE

1

Start

所有 candidate 都先保留

2

Shuffle Cases

隨機打亂 7 個 opponent cases

3

Best Score

看當前 case 的最高分

4

Filter

只保留該 case 分數 = 最高分的 candidate

5

Repeat / Tie-break

剩 1 個就選出；若全部 case 跑完仍平手，隨機選 1 個

Current EAGLE 重點

- selection case = 7 個 opponent scores
- 沒有 epsilon / tolerance
- 每次選父代都重新隨機 case 順序
- parent A、parent B 各自獨立做一次 lexicase

Why useful?

- 保留能在特定 opponent 表現突出的 specialist
- 增加族群策略多樣性

Strategy Reflection Roles

用途與 *prompt focus*

1 Match Commentator

Purpose	分析單場對戰： 雙方策略、轉折點、勝敗原因
Prompt focus	input: 1 場 selected match + tick log + strategy prompt

2 Manager

Purpose	整合多場分析，找 recurring weaknesses / strengths
Prompt focus	input: aggregate game evidence + selected analyses

3 Coach

Purpose	把分析轉成新的 strategy prompt 與具體規則
Prompt focus	input: parent strategy prompt + manager plan + mutation intent

4 Generator

Purpose	依新 prompt 產生完整 CandidateAgent.java
Prompt focus	input: strategy prompt + generation prompt + previous code

selected matches → Commentator → Manager → Coach → new strategy prompt → Generator

Selection policy

- 最多取 3 場做 reflection
- loss > draw > win（嚴格優先，不是加權抽樣）
- 用完的 raw logs 之後會刪除，只保留 summary / analysis

Role difference

- 前三個 role 負責「分析與改 prompt」
- Generator 才負責「真的生成 Java」

Strategy Reflection Roles

用途與 *prompt focus*

目前 active path 沒有 *Manager* role

1 Match Commentator

Purpose	分析單場對戰： 雙方策略、轉折點、勝敗原因
Prompt focus	input: 1 場 selected match + tick log + strategy prompt

2 Coach

Purpose	整合多場分析與 summary， 產出新的 strategy prompt
Prompt focus	input: aggregate game evidence + selected analyses + current strategy prompt

3 Code Reflection

Purpose	分析 source / compiler / runtime evidence，找出 code 問題與修正方向
Prompt focus	input: previous code + error / warning / match evidence + generation prompt

4 Generator

Purpose	依 strategy prompt + generation prompt + previous code 產生完整 CandidateAgent.java
Prompt focus	input: strategy prompt + generation prompt + previous code

selected matches → Commentator → Coach → new strategy prompt → Generator

Selection policy

- 最多取 10 場做 reflection
- coverage-aware：先涵蓋 opponent，再補 map / side coverage
- raw logs 之後會刪除，只保留 summary / analysis

Role difference

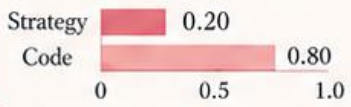
- Commentator / Coach 負責策略變異（strategy mutation）
- Code Reflection 負責程式變異（code mutation evidence）
- Generator 才負責真的生成 Java

AOS (Adaptive Operator Selection) 怎麼做

Current EAGLE operator choice and update rule

1 初始偏好 Initial probabilities

Strategy Mutation = 0.20
Code Mutation = 0.80
minimum probability = 0.10



2 選擇 Operator Choose operator

依目前 probability 抽選
Strategy 或 Code

3 產生 Offspring Apply mutation

Strategy: Commentator → Coach

Code: Reflection → Rewrite

4 完整評估 Evaluate offspring

Java generation → validation
→ compilation
→ 126 MicroRTS matches
→ 7 opponent scores

5 計算 Reward Compare with parent

execution-first
再看 7 個 opponent 的
Win / Draw / Loss rank 改變

6 更新偏好 EMA update

每代更新 operator preference
EMA $\alpha = 0.20$
每個 operator 至少保留 0.10

AOS 重點

- 只有 2 種 operator：Strategy Mutation / Code Mutation
- 一開始偏向 Code（0.80），因為先穩定生成與執行
- 不會讓其中一個 operator 降到 0
- Code Quality 不參與 AOS reward

Reward 依據

- 先看 offspring 是否成功執行
- 再看相對 parent 的射手表現變化
- 較 7 個 opponents 的 W / D / L
  rank improvement 計算 reward
- 用 reward 更新下一代 operator preference

EMA 計算公式

$$Q_{new} = (1 - \alpha)\, Q_{old} + \alpha\, R$$

- Q：operator quality estimate
- R：本代 reward
- 目前 $\alpha = 0.20$

例如：$0.8 \times Q_{old} + 0.2 \times R$

兩個 operator 的 probability 合計為 1.0，每個至少保留 0.10

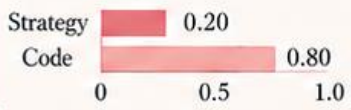
AOS (Adaptive Operator Selection) 怎麼做

Current EAGLE operator choice and update rule

1

初始偏好
Initial probabilities

Strategy Mutation = 0.20
Code Mutation = 0.80
minimum probability = 0.10



2

選擇 Operator
Choose operator

依目前 probability 抽選
Strategy 或 Code

3

產生 Offspring
Apply mutation

Strategy: Commentator → Coach

Code: Reflection → Rewrite

4

完整評估
Evaluate offspring

Java generation → validation
→ compilation
→ 126 MicroRTS matches
→ 7 opponent scores

5

父子對戰
Head-to-head reward

child vs comparison parent
3 maps × 3 rounds × p0/p1
= 18 matches
win = 1, draw = 0.5, loss = 0
reward = (wins + 0.5 × draws) / 18

6

更新偏好
EMA update

每代更新 operator preference
EMA $\alpha = 0.20$
每個 operator 至少保留 0.10

AOS 重點

- 只有 2 種 operator：Strategy Mutation / Code Mutation
- 一開始偏向 Code（0.80），因為先穩定生成與執行
- 不會讓其中一個 operator 降到 0
- Code Quality 不參與 AOS reward

Reward 依據

- 以 offspring 與比較 parent 的直接對戰結果計分
- 共 18 場（3 maps × 3 rounds × p0/p1）
- 勝 = 1，和 = 0.5，敗 = 0
- reward = (wins + 0.5 × draws) / 18
- 用 reward 更新下一代 operator preference

EMA 計算公式

$$Q_{new} = (1 - \alpha)\, Q_{old} + \alpha\, R$$

- Q：operator quality estimate
- R：本代 reward
- 目前 $\alpha = 0.20$

例如：$0.8 \times Q_{old} + 0.2 \times R$

兩個 operator 的 probability 合計為 1.0，每個至少保留 0.10